Lab Handout 3-H

Dataset Merging, Exploratory Data Analysis, and Module 4 Handoff

Capstone — Module 3: System Integration and Data Collection

1 Objective

Merge all campaign CSV files into a single, clean, labeled dataset; perform the five required exploratory data analysis (EDA) visualizations; document the operating-condition shift between the clean regime (C1, C2, and C4 single-station DATA-only) and the contended regime (C5 multi-station ping-pong), entirely on hardware data; and produce the data dictionary that serves as the Module 4 data contract. The merged dataset is the primary deliverable of Module 3.

2 Prerequisites

- Lab Handout 3-G complete (campaigns C1, C2, C4, and C5 executed and validated).
- Python with `pandas`, `matplotlib`, `seaborn`, and `scipy` installed.

3 Procedure

3.1 Step 1 — Merging script

Place `merge_dataset.py` in the `Code/Tools/CampaignCollect/` folder alongside `collect.py`. The script auto-discovers every `results_*.csv` under the `logs/` subdirectory, so no file list needs to be maintained as new campaigns are added. Run it with `python merge_dataset.py`; it writes `merged_dataset.csv` in the same folder.

```python
import csv
import sys
import random
from pathlib import Path
from collections import Counter, defaultdict

SCRIPT_DIR = Path(__file__).parent
LOGS_DIR   = SCRIPT_DIR / "logs"
OUT_CSV    = SCRIPT_DIR / "merged_dataset.csv"

EXPECTED_COLS = [
    "campaign_id", "run_id", "timestamp_host",
    "rx_timestamp_ms", "tx_timestamp_ms", "inter_arrival_ms",
    "beacon_id", "seq_num", "pkt_type", "mod_kind", "spread_factor",
    "rssi_dbm", "snr_db", "freq_error_hz", "payload_len",
    "distance_m", "label_sf", "label_mod", "mod_kind_label",
    "label_beacon", "label_pkt",
```

```python
]

SKIP_DIRS  = {"dry_run"}
SKIP_FILES = {"C1_combined.csv", "C2_combined.csv", "merged_dataset.csv"}


def discover_csvs():
    found = []
    for p in sorted(LOGS_DIR.rglob("results_*.csv")):
        if p.parent.name in SKIP_DIRS:
            continue
        if p.name in SKIP_FILES:
            continue
        found.append(p)
    return found


def stratified_split(rows, label_col, fracs=(0.70, 0.15, 0.15), seed=42):
    # Split at run_id level stratified by label_col to prevent leakage:
    # rows within one run share channel state, so splitting rows randomly
    # would leak that context into the test set.
    rng = random.Random(seed)
    by_label = defaultdict(lambda: defaultdict(list))
    for i, row in enumerate(rows):
        by_label[row[label_col]][row["run_id"]].append(i)

    split_map = {}
    for label_val, run_dict in by_label.items():
        runs = sorted(run_dict.keys())
        rng.shuffle(runs)
        n = len(runs)
        n_train = max(1, round(n * fracs[0]))
        n_val   = max(1, round(n * fracs[1]))
        for j, run_id in enumerate(runs):
            tag = ("train" if j < n_train
                   else "val" if j < n_train + n_val
                   else "test")
            for idx in run_dict[run_id]:
                split_map[idx] = tag
    return split_map


csvs = discover_csvs()
if not csvs:
    print(f"No results_*.csv files found under {LOGS_DIR}")
    sys.exit(1)

print(f"Found {len(csvs)} campaign CSV(s):\n")
all_rows = []
for p in csvs:
    with open(p, newline="") as f:
        rows = list(csv.DictReader(f))
    if not rows:
        print(f"  [SKIP] {p.name} - empty")
```

```
        continue
    missing = set(EXPECTED_COLS) - set(rows[0].keys())
    if missing:
        print(f"  [WARN] {p.name} missing columns: {missing}")
    cid = rows[0].get("campaign_id", "?")
    rid = rows[0].get("run_id", "?")
    print(f"  {p.name}  ->  campaign={cid}  run={rid}  rows={len(rows)}")
    all_rows.extend(rows)

print(f"\nTotal rows before dedup: {len(all_rows)}")

seen = set()
deduped = []
for row in all_rows:
    key = (row.get("run_id"), row.get("beacon_id"),
           row.get("seq_num"), row.get("pkt_type"))
    if key not in seen:
        seen.add(key)
        deduped.append(row)
print(f"Total rows after dedup:  {len(deduped)}")

for row in deduped:
    row["regime"] = (
        "contended" if row.get("campaign_id", "").startswith("C5")
        else "clean")

split_map = stratified_split(deduped, "label_sf")
for i, row in enumerate(deduped):
    row["split"] = split_map.get(i, "train")

by_campaign = Counter(r["campaign_id"] for r in deduped)
by_regime   = Counter(r["regime"]      for r in deduped)
by_pkt      = Counter(r["pkt_type"]    for r in deduped)
by_split    = Counter(r["split"]       for r in deduped)

print("\n--- Summary ---")
print("Rows per campaign_id:")
for k, v in sorted(by_campaign.items()):
    print(f"  {k}: {v}")
print(f"Regime:   {dict(by_regime)}")
print(f"pkt_type: {dict(by_pkt)}")
print(f"Split:    {dict(by_split)}")

out_cols = EXPECTED_COLS + ["regime", "split"]
with open(OUT_CSV, "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=out_cols, extrasaction="ignore")
    writer.writeheader()
    writer.writerows(deduped)

print(f"\nSaved: {OUT_CSV}  ({len(deduped)} rows)")
```

Listing 1: Dataset merging script (`merge_dataset.py`).

Stratification at the run_id Level

Split at the *run_id* level, not at `campaign_id` or at the individual-sample level. Samples inside one run share channel conditions and hardware state; splitting samples randomly leaks channel context into the test set. Splitting on `campaign_id` alone is also unsafe: one campaign family can contain several distinct collection runs. Keying on `run_id` (one value per invocation of `collect.py`) is the only way to keep splits genuinely independent.

Canonical Modulation Kind and Packet Type Mappings

Modulation kind is represented throughout Modules 1–5 using the integer codes **1=LoRa, 2=FSK, 3=OOK** (Module 1 Table 4). The `mod_kind` feature vector byte, the `mod_kind_label` config column, and any Module 4 integer checks use this mapping. The string alias `label_mod` ("lora"/"fsk"/"ook") is retained only for readability and never used in a numeric check. Do not introduce a 0/1 alias.

Packet type is **0=DATA, 1=ACK, 2=PAUSE**. `pkt_type` is extracted from the low 2 bits of the radio packet's `Modulation Code` byte by the LH2-F bit-field extractor and is the canonical authority for the `label_pkt` ground-truth label.

3.2 Step 2 — Verify merged dataset statistics

After running `merge_dataset.py`, confirm:

- Total rows $\geq$ 2,100 (C1: 800, C2: 800, C4: 800, C5: $\sim$600 = 3000 theoretical; expect some Packet Error Rate (PER)-driven shortfall and C4 may be partial depending on execution).
- Rows per campaign ID roughly as expected from the target packet counts.
- `regime` column shows roughly 75% `clean` (C1, C2, C4) and 25% `contended` (C5); the LH4-F operating-condition shift experiment relies on having both regimes well-represented.
- `pkt_type` value counts: `0` (DATA) dominates due to C1–C4; `1` (ACK) appears only in C5b.
- `split` column contains "train", "val", and "test" at approximately 70/15/15%.
- No `[WARN]` lines in the script output indicating missing schema columns. The script does not compute a `label_mismatch` column; verify firmware config correctness by checking that the `spread_factor` column matches `label_sf` for all DATA rows (`pkt_type = 0`).

3.3 Step 3 — EDA Visualization 1: RSSI by TX power (C1)

```python
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

df = pd.read_csv("merged_dataset.csv")
c1 = df[df["campaign_id"] == "C1"]
# sweep_value (TX power) is not stored as a column; use run_id to recover
    it.
c1 = c1.copy()
c1["tx_power"] = c1["run_id"].str.extract(r"pwr(\d+)").astype(int)
sns.violinplot(data=c1, x="tx_power", y="rssi_dbm")
plt.xlabel("TX Power (dBm)")
plt.ylabel("RSSI (dBm)")
plt.title("Campaign C1: RSSI Distribution by TX Power")
plt.savefig("eda1_rssi_by_txpower.png", dpi=150)
```

Expected: distributions shift right (higher RSSI, less negative) as TX power increases. The distributions should overlap slightly at adjacent power levels, indicating the system is sensitive to 6 dB power steps but not perfectly separable by power alone.

3.4 Step 4 — EDA Visualization 2: RSSI vs. distance with FSPL overlay (C4)

```python
import numpy as np

c4 = df[df["campaign_id"] == "C4"]
means = c4.groupby("distance_m")["rssi_dbm"].mean()
stds  = c4.groupby("distance_m")["rssi_dbm"].std()

# FSPL model: RSSI(d) = P_tx - 20*log10(d) - 20*log10(f_Hz) + 147.55
f_hz  = 906.5e6
p_tx  = 14  # dBm
```

```
d_range = np.linspace(0.5, 10, 200)
fspl = p_tx - 20*np.log10(d_range) - 20*np.log10(f_hz) + 147.55

plt.figure()
plt.errorbar(means.index, means.values, yerr=stds.values,
             fmt="o", capsize=5, label="Measured mean RSSI")
plt.plot(d_range, fspl, "--", label="FSPL model (+14 dBm, 906.5 MHz)")
plt.xlabel("Distance (m)")
plt.ylabel("RSSI (dBm)")
plt.title("Campaign C4: RSSI vs. Distance")
plt.legend()
plt.savefig("eda2_rssi_vs_distance.png", dpi=150)
```

3.5 Step 5 — EDA Visualization 3: Feature correlation matrix

```
numeric_cols = ["rssi_dbm", "snr_db", "inter_arrival_ms",
                "freq_error_hz", "spread_factor", "distance_m"]
corr = df[numeric_cols].astype(float).corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Feature Correlation Matrix (Full Dataset)")
plt.tight_layout()
plt.savefig("eda3_correlation_matrix.png", dpi=150)
```

Identify and discuss in your report: which feature pairs are strongly correlated ($|r| > 0.7$), and whether any correlations might cause problems for a linear classifier.

3.6 Step 6 — EDA Visualization 4: Inter-arrival time by station count (C5a vs. C5b)

```
c5a = df[df["run_id"].str.startswith("C5a")]["inter_arrival_ms"]
c5b = df[df["run_id"].str.startswith("C5b")]["inter_arrival_ms"]
plt.figure()
plt.hist(c5a, bins=40, alpha=0.6, label="C5a Single station (DATA-only)")
plt.hist(c5b, bins=40, alpha=0.6, label="C5b Ping-pong (DATA + ACK)")
plt.xlabel("Inter-arrival Time (ms)")
plt.ylabel("Count")
plt.title("Inter-arrival Time: Single Station vs. Ping-pong")
plt.legend()
plt.savefig("eda4_interarrival_scheduling.png", dpi=150)
```

Expected: C5a produces a tight peak near 2 s (fixed initiator interval); C5b produces a bimodal distribution — a DATA peak near 1 s and a tighter ACK peak near the round-trip latency. The ACK cluster is a learnable temporal signature for emitter identification in Module 4.

3.7 Step 7 — EDA Visualization 5: Class balance (4 tasks)

```
PKT_LABELS = {0: "DATA", 1: "ACK"}
TASKS = [
    ("label_sf",     "Spreading Factor",  str),
    ("label_mod",    "Modulation",        str),
    ("label_beacon", "Beacon ID",         str),
    ("label_pkt",    "Packet Type",       lambda x: PKT_LABELS.get(int(x),
        str(x))),
]
fig, axes = plt.subplots(1, 4, figsize=(14, 4))
for ax, (col, title, fmt) in zip(axes, TASKS):
    counts = df[col].value_counts().sort_index()
    labels = [fmt(k) for k in counts.index]
    bars = ax.bar(labels, counts.values, color="steelblue", edgecolor="
        white")
    ax.set_title(title)
    ax.set_xlabel(col)
    ax.set_ylabel("Count")
    for bar, v in zip(bars, counts.values):
        if v < 150:
            bar.set_color("tomato")
plt.tight_layout()
plt.savefig("eda5_class_balance.png", dpi=150)
```

Bars drawn in red indicate fewer than 150 samples; the 70/15/15 split will produce fewer than 30 test samples for that class, which is insufficient for reliable evaluation.

3.8 Step 8 — Operating-condition shift analysis

Compare at least two features between the clean regime (C1, C2, and C4 single-station, DATA-only) and the contended regime (C5 cooperative-target ping-pong), entirely within the hardware dataset. The `regime` column added by the merge script identifies the source for each row.

1. **RSSI shift:** Compute mean and standard deviation of RSSI for equivalent baseline conditions (SF7, +14 dBm, ∼3 m), comparing C1's +14 dBm rows against the DATA rows of C5b. Half-duplex shadowing during ACK exchanges produces correlated dropouts that change RSSI variance even though the channel is physically the same.

2. **Inter-arrival time shift:** Compare inter-arrival time histograms between the clean regime (tightly clustered ∼2 s) and the contended regime (multi-modal: ∼1 s DATA cadence plus ACK round-trip).

3. For each comparison, state: the direction of the shift (e.g., "contended-regime RSSI std is 1.5× the clean-regime value"), the likely physical cause (half-duplex shadowing, ACK collisions), and the implication for Module 4 (*"a classifier trained only on the clean regime may misclassify ACK rows as DATA when deployed on contended traffic, leading to lower packet-type accuracy than the clean-only test set suggests"*).

Why Operating-Condition Shift Matters for Module 4

Module 4 will train classifiers on the hardware dataset collected here and evaluate them on held-out hardware data. The LH4-F experiment partitions the data along the `regime` axis: train on the clean regime (C1–C4), evaluate on the contended regime (C5), and measure the accuracy drop $\Delta = \text{Acc}(\text{clean} \rightarrow \text{clean}) - \text{Acc}(\text{clean} \rightarrow \text{contended})$. The distribution shift documented in this step provides the empirical explanation for that drop. Without this analysis, the Module 4 operating-condition shift experiment has no interpretive foundation.

3.9 Step 9 — Data dictionary

Write a `data_dictionary.md` file documenting every column in `merged_dataset.csv`:

Column	Type	Units / Valid Range	Description
`campaign_id`	string	C1, C2, C4, C5	Experiment family identifier
`run_id`	string	unique per run	One `collect.py` invocation (ISO-timestamped) — split-leakage key
`regime`	string	"clean"/"contended"	Operating regime (clean = C1, C2, C4; contended = C5)
`timestamp_host`	ISO 8601	UTC	Host reception time
`rx_timestamp_ms`	uint32	ms	Interceptor-side STM32 tick at RX
`tx_timestamp_ms`	uint32	ms	Transmitter-side STM32 tick (from packet payload)
`inter_arrival_ms`	int32	ms	RX timestamp delta between consecutive packets
`beacon_id`	uint8	1–255	Sender Beacon ID (`0x01`=Target A, `0x02`=Target B, `0xFF`=Threat)
`seq_num`	uint16	≥ 0	Per-station sequence counter (resets on firmware flash)
`pkt_type`	uint8	0=DATA, 1=ACK	Packet type from low 2 bits of `Modulation Code` byte
`mod_kind`	uint8	1=LoRa, 2=FSK, 3=OOK	Self-reported modulation kind (upper 6 bits of `Modulation Code` byte)

Column	Type	Units / Valid Range	Description
`spread_factor`	uint8	7–10 (LoRa); 0 (non-LoRa)	Self-reported SF (fixed by firmware compile-time define)
`rssi_dbm`	int8	−120 to 0 dBm	Received signal strength at interceptor
`snr_db`	float	−20 to +15 dB	SNR estimate (SX1276 `RegPktSnrValue` × 0.25)
`freq_error_hz`	int16	Hz	Crystal frequency offset between TX and interceptor
`payload_len`	uint8	bytes	Radio payload length byte from packet
`distance_m`	float	0.5–10 m	Measured Target–interceptor antenna separation
`label_sf`	uint8	7–10	Ground-truth spreading factor (from campaign config)
`label_mod`	string	"lora"	Human-readable modulation label (alias of `mod_kind_label`)
`mod_kind_label`	uint8	1=LoRa	Ground-truth modulation kind integer (from campaign config)
`label_beacon`	uint8	1 or 2	Ground-truth Beacon ID (sender identity, from packet)
`label_pkt`	uint8	0 or 1	Ground-truth packet type (from packet)
`split`	string	"train"/"val"/"test"	Dataset partition (stratified by `label_sf`, keyed on `run_id`)

4 Verification Checklist

- ☐ `merged_dataset.csv` contains ≥2,100 rows with the correct schema and a `split` column.
- ☐ No `[WARN]` lines in merge script output; `spread_factor` matches `label_sf` for all DATA rows.
- ☐ Both regimes well-represented: `regime` = `clean` and `regime` = `contended` both present with non-trivial row counts.
- ☐ All four classification tasks (SF, modulation, Beacon ID, packet type) have at least 30 test samples per class.
- ☐ Train/val/test split is approximately 70/15/15%.
- ☐ All five EDA plots generated and saved as `.png` files (Visualization 5 now covers all four classification tasks).
- ☐ Operating-condition shift analysis includes at least two quantitative comparisons (clean vs. contended mean/std or histogram divergence) entirely on hardware data.
- ☐ Data dictionary covers all columns with units and valid ranges, including the `pkt_type` and `regime` columns.

Lab Handout 3-H Deliverable — and Module 3 Final Deliverable

1. `merged_dataset.csv`: the complete labeled dataset with stratified split column.
2. `merge_dataset.py`: the merging script.
3. Dataset summary: total rows, rows per campaign, rows per regime (clean / contended), rows per (Beacon ID, `pkt_type`) pair, rows per class for all four classification tasks (SF, modulation, Beacon ID, packet type).
4. All five EDA plots (`eda1_` through `eda5_`), each with a 2–3 sentence interpretive caption.
5. Operating-condition shift analysis: at least two quantitative comparisons between the clean regime (C1, C2, and C4) and the contended regime (C5) with physical interpretation.
6. `data_dictionary.md`: complete column documentation.
7. Known limitations section: indoor multipath effects, distance measurement uncertainty, half-duplex shadowing during Threat RX, any collection anomalies (power interruptions, firmware crashes, station desynchronization), and any campaigns that fell short of their target packet count.
8. One-page Module 3 retrospective: what worked, what integration failures were most instructive, and what you would change before a second data collection campaign.

Glossary

PER	Packet Error Rate	5